Hackathon Use Case: Voucher Flash Sale System

1. Business background

PromoRush is a digital promotion platform partnering with merchants across Indonesia. To celebrate its anniversary, PromoRush will launch a nationwide **"Mega Voucher Flash Sale"**, allowing customers to purchase shopping vouchers at significantly discounted prices.

The campaign covers four merchant categories:

- Food & Beverage
- Fashion & Lifestyle
- Electronics

PromoRush expects **5 million visitors throughout the campaign**, with up to **100,000 users attempting to purchase within the first minute of a flash-sale session**. However, only **100,000 vouchers** are available across the entire campaign.

Your team has been selected to build a system that handles this demand while ensuring accurate stock allocation, reliable payment processing, and a clear customer experience.

2. Voucher allocation and pricing

Each voucher provides a fixed discount when redeemed at an eligible merchant.

Category	Voucher discount value	Flash-sale purchase price	Minimum merchant transaction	Total quota
Food & Beverage	Rp50,000	Rp10,000	Rp100,000	50,000
Fashion & Lifestyle	Rp200,000	Rp50,000	Rp400,000	40,000
Electronics	Rp500,000	Rp150,000	Rp1,000,000	10,000
Total				**100,000**

Additional business rules:

- Vouchers are valid only at participating merchants within their category.
- Vouchers cannot be exchanged for cash or transferred to another customer.
- Each voucher can be redeemed once.
- Only one voucher may be applied to a merchant transaction.
- Any unused discount value is forfeited and not refundable.

3. Flash-sale schedule

The campaign runs for four consecutive Saturdays, with one session each Saturday from **12:00 to 12:15 WIB**.

Each session receives **25% of each category's total quota**.

Category	Quota per session
Food & Beverage	10,000
Fashion & Lifestyle	9,000
Electronics	6,000
Total	**25,000**

The system must enforce these rules:

- Customers can browse vouchers before a session begins.
- New purchase reservations are accepted only during the active session.
- The server determines the official opening and closing times.
- Unsold vouchers do not carry over to another session.
- A valid reservation created before closing may still be paid before its own payment deadline.
- Released stock becomes available again only while the original session remains open.

4. Customer eligibility and purchase limits

Customers must sign in using an account with a verified phone number.

For this exercise, the verified phone number represents a unique customer identity; actual SMS delivery may be simulated.

Each customer:

- May purchase **one voucher per category across the entire campaign**.
- May therefore purchase a maximum of four vouchers.
- May hold only one active purchase reservation per category.
- May retry after a reservation expires or a payment definitively fails, provided the session is still open and stock remains.

The system must enforce these limits when requests arrive from multiple tabs, devices, or simultaneous API calls.

5. Purchase and stock reservation

The customer selects a voucher and starts checkout.

If the customer is eligible and stock is available, the system reserves **one voucher for five minutes** and creates a pending order.

During this period:

- Reserved stock cannot be allocated to another customer.
- The customer sees a payment deadline.
- Refreshing the page must recover the existing order.
- Repeated purchase requests must not create duplicate reservations.

If payment succeeds within the reservation deadline, the purchase is confirmed.

If payment definitively fails or the reservation expires, the stock and customer entitlement are released.

Critical requirement: confirmed purchases plus active reservations must never exceed the category's allocated session quota.

6. Payment integration

Customers can choose between two simulated payment gateways:

- Midtrans
- Xendit

Both integrations use **stub services**, so no real payment credentials or transactions are required.

The payment simulation must support:

- Successful payment
- Failed payment
- Pending payment

Repeated payment callbacks must not create additional vouchers or duplicate payment records

For a deterministic training rule, a success callback processed after the reservation expires must **not** issue a voucher. The payment must instead be recorded as **refund required**, with the refund process simulated.

Participants must also explain how payment confirmation and reservation expiry are handled when they occur simultaneously.

7. Voucher issuance and customer wallet

After a purchase is confirmed, the customer receives a digital voucher containing:

- Unique voucher identifier and redemption code
- QR code
- Merchant category
- Discount value
- Minimum transaction amount
- Issue and expiry timestamps
- Current voucher status

Vouchers remain valid for **seven days from issuance**.

Customers can access their vouchers through **My Vouchers**, with the following states:

Status	Meaning
Active	Available for redemption
Redeemed	Successfully used
Expired	Validity period has ended

A downloadable voucher PDF is an optional enhancement.

If payment succeeds but voucher generation temporarily fails, the system must recover and issue exactly one voucher. The customer should see that fulfilment is still processing.

8. Merchant redemption

A merchant-facing page or API allows an authenticated merchant to scan or enter a voucher code and submit the transaction amount.

Before redemption, the system validates that:

- The voucher exists and belongs to the purchasing customer.
- The merchant is eligible for the voucher category.
- The voucher has not expired.
- The minimum transaction amount is satisfied.
- The voucher has not already been redeemed.

A successful redemption records the merchant, transaction reference, transaction amount, discount applied, and redemption timestamp.

Critical requirement: if two redemption requests arrive simultaneously for the same voucher, only one may succeed.

For the exercise, merchant checkout and customer identification can be simulated.